EXAM CRASH COURSE

Compiler Construction

Core Concepts, Seven Pipeline Phases, Syntax Trees

TARGET

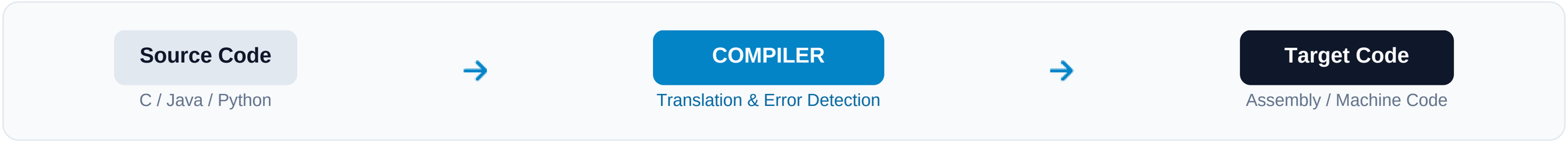
Semester Exam Prep

FORMAT

Theory + Worked Examples

PHASE 0 • FOUNDATIONS

Introduction to Compiler Construction



What is a Compiler?

A special program that converts high-level source code into low-level machine code in a single execution flow.

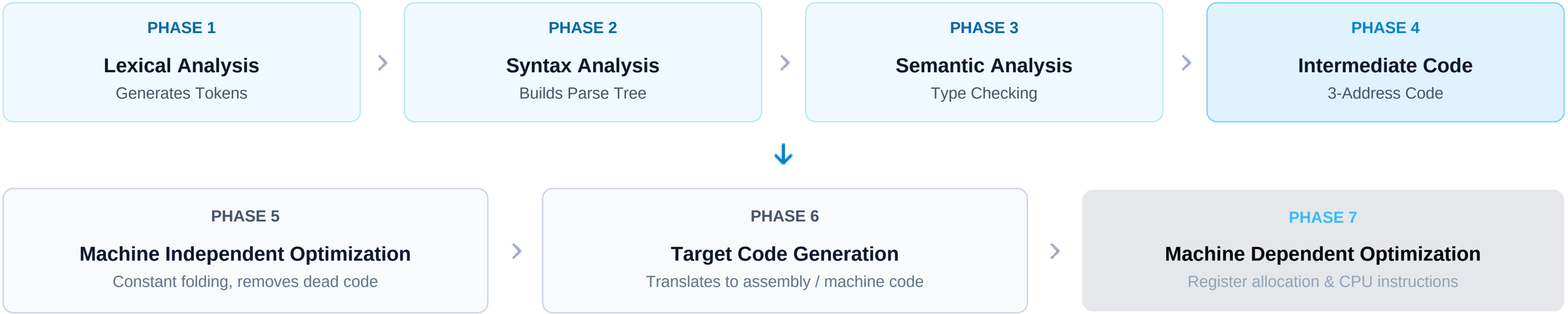
- **Speed:** Fast runtime execution after compilation.
- **Security:** Source code is not distributed to end users.

Compiler vs. Interpreter

Feature	Compiler	Interpreter
Translation	Entire program at once	Line-by-line
Execution Speed	Significantly faster	Slower
Error Detection	Lists all errors together	Stops at first error

ARCHITECTURE • SEVEN PHASES

Seven Phases of a Compiler



PHASE 1 • WORKED EXAMPLE

Lexical Analysis: Step-by-Step Walkthrough

SOURCE STATEMENT: `int sum = a + 10;`

Lexeme	Token Type	Pattern / Description Rule
int	Keyword	Predefined reserved language word
sum	Identifier	Letter followed by alphanumeric characters
=	Assignment Operator	Literal assignment symbol (=)
a	Identifier	Single letter identifier
+	Arithmetic Operator	Literal addition symbol (+)
10	Integer Constant	One or more decimal digits ([0-9]+)
;	Separator / Punctuator	Statement delimiter (;)

TOKEN OUTPUT: `(KEYWORD, int) (IDENTIFIER, sum) (ASSIGN_OP, =) (IDENTIFIER, a) (PLUS_OP, +) (NUMBER, 10) (SEMICOLON, ;)`

PHASE 1 • REFERENCE

Standard Token Types in Programming

Token Type	Definition & Purpose	Examples
Keyword	Reserved word with predefined syntax meaning	int, if, else, while, return
Identifier	User-defined names for variables, functions, or arrays	sum, age, main, counter
Operator	Symbols triggering mathematical or logical operations	+, -, *, /, =, ==, <=
Constant / Literal	Fixed immutable numeric, character, or boolean values	10, 3.14, 'A', true
Separator	Punctuation structuring statements and blocks	; , ( ) { } [ ]
String Literal	Sequences of characters enclosed in quotes	"Hello", "Result: "
Comments	Descriptive notes stripped by lexer (never reach parser)	// single line, /* block */

PHASE 2 • PARSING

Syntax Analysis (The Parser)

INPUT Tokens from Lexical Analyzer → Checked against **Grammar Rules** → Produces **Parse Tree / AST**

Role of the Parser

Validates whether the incoming token stream conforms to the formal syntax rules of the programming language.

Grammar Rules

Defined using **Context-Free Grammars (CFG)**. Dictates valid statements, loops, and expression structures.

Syntax Errors

Triggered when rules are violated. E.g., missing semicolon `int a = 5` or unmatched parenthesis `(a + b*c`.

Exam Tip: The lexer catches invalid characters (e.g., int @x;), but the parser catches invalid statement structure (e.g., + = a b;).

PHASE 2 • GRAMMAR FUNDAMENTALS

Terminals vs. Non-Terminals

CAN EXPAND

Non-Terminals

Syntactic variables that can be replaced or expanded further using grammar production rules.

Usually written in **UPPERCASE** letters (e.g., **E, T, F, S**).

CANNOT EXPAND

Terminals

The basic symbols or tokens that cannot be broken down further. They form the actual source text.

Usually written in **lowercase**, numbers, or symbols (e.g., **id, +, *, num**).

Grammar Example:

```
E → E + T | T
T → id
```

→ **Non-Terminals:** , E T

→ **Terminals:** , + id

PHASE 2 • TREES & GRAMMARS

Context-Free Grammar & Parse Tree

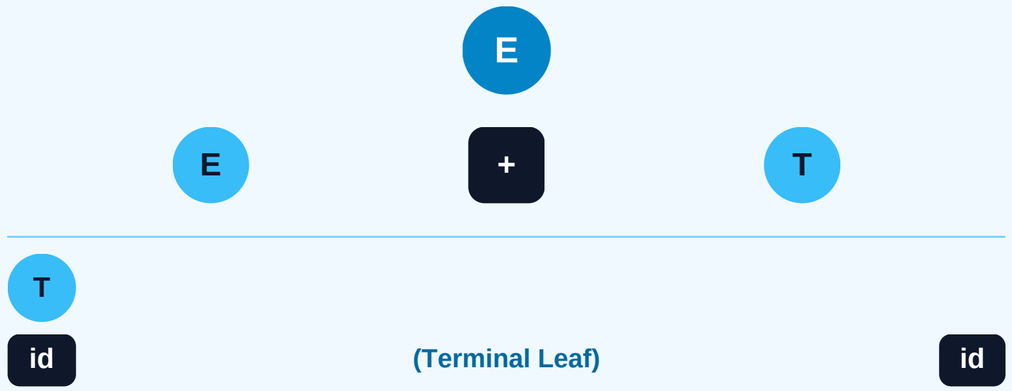
What is a CFG?

A formal set of production rules of the form **A → α** where A is a single non-terminal and α is a string of terminals and/or non-terminals.

Parse Tree Properties

- **Root Node:** The start symbol of the grammar (E).
- **Interior Nodes:** Always **Non-Terminals**.
- **Leaf Nodes:** Always **Terminals** (read left-to-right).

PARSE TREE FOR: id + id



PHASE 3 • TYPE & MEANING CHECKING

Semantic Analysis (Context & Types)

Primary Role: Checks whether the code makes logical sense according to the programming language definitions. A statement can be syntactically valid yet semantically completely wrong!

Core Responsibilities

- **Type Checking:** Verifies compatibility (e.g., adding float to pointer).
- **Scope Checking:** Ensures variables are declared before use.
- **Array Bounds / Signatures:** Validates correct function parameter counts.

Classic Exam Example

```
int age;
 age = "twenty five"; // ERROR!
```

Why is this a semantic error?

Syntax is 100% correct (variable = value;), but semantics fail because a string literal cannot be assigned to an integer variable.

PHASE 4 • UNIVERSAL REPRESENTATION

Intermediate Code Generation (ICG)

Why Generate Intermediate Code?

It sits between the front-end (source language) and back-end (machine target). It is easy to generate and easy to translate into target machine code.

- **Portability:** Allows retargeting to new CPUs with zero changes to front-end.
- **Optimization:** Enables architecture-independent code improvements.

Three-Address Code (TAC)

EXAM FAVORITE

Each instruction has at most one operator on the right-hand side:

Source Expression:

```
x = a + b * c
```

GENERATED 3-ADDRESS CODE: t1 = b * c t2 = a + t1 x = t2

```
t1 = b * c t2 = a + t1 x = t2
```

PHASE 5 • CPU-AGNOSTIC OPTIMIZATION

Machine Independent Code Optimization

Improves intermediate code so that execution is faster and uses less memory, without considering any specific processor or target CPU hardware.

TECHNIQUE 1

Constant Folding

Evaluating constant expressions at compile-time instead of runtime.

```
Original: x = 2 * 3
Optimized: x = 6
```

TECHNIQUE 2

Dead Code Elimination

Removing code that never executes or has no effect on program output.

```
Original: if (0) { y = 10; }
Optimized: (Entire block removed)
```

TECHNIQUE 3

Subexpression Elimination

Avoiding redundant recalculation of identical expressions.

```
Original: a = x + y; b = x + y;
Optimized: t1 = x + y; a = t1; b =
t1;
```

PHASES 6 & 7 • TARGET GENERATION & HARDWARE TUNING

Code Generation & Target Optimization

PHASE 6

Target Code Generation

Translates the intermediate code representation into relocatable machine code or target assembly instructions.

SAMPLE ASSEMBLY EMISSION:

```
MOV R1, b
 MUL R1, c
 ADD R1, a
 MOV x, R1
```

PHASE 7

Machine Dependent Optimization

Fine-tunes the target instructions based on intimate knowledge of the specific target CPU architecture.

- **Register Allocation:** Keeping hot variables in CPU registers instead of slow RAM.
- **Instruction Selection:** Using specialized CPU instructions (e.g., SIMD, hardware multipliers).
- **Peephole Optimization:** Replacing short instruction sequences with single faster ones.

COMPARISON • HIGH-YIELD TABLE

Comparison of Optimization Phases

Parameter	Machine Independent Optimization	Machine Dependent Optimization
Pipeline Stage	Applied to **Intermediate Code (Phase 5)**	Applied to **Target Machine Code (Phase 7)**
Hardware Dependency	Completely independent of CPU architecture	Tailored strictly to target CPU registers & flags
Primary Techniques	• Dead Code Elimination	• Peephole Optimization
Portability Benefit	Reusable across all target processor architectures	Must be rewritten for each new hardware family (e.g. x86 vs ARM)

FINAL CHECKPOINT

Exam Practice MCQs & Final Recap

Fast Recall Check

Lexer: Characters → Tokens

Parser: Tokens → Parse Tree

Semantics: Types & Valid Meaning

Q1: Which phase removes white space and comments? — Lexical Analysis

Q2: In a parse tree, leaf nodes are always? — Terminals

Q3: Replacing x = 2 * 3 with x = 6 at compile time is called? — Constant Folding

Q4: Type checking and scope checking are performed in? — Semantic Analysis

Q5: Register allocation belongs to which optimization phase? — Machine Dependent

All the best for your exam on Saturday! Best of luck with your Google Meet review session.